r_tutorial_week_04

Werkcollege 4

Sebastian A. J. Lortz

15 October 2026

Table of contents

How to use this tutorial **2**

1 Start workflow **3**
Your turn . . . 3

2 From estimating to testing **3**

3 The five steps of a hypothesis test **4**

4 One-sample t-test: comparing our mean to a known value **4**
4.1 Step 1 – Hypotheses . . . 4
Your turn . . . 5
4.2 Step 2 – Assumptions . . . 5
Your turn . . . 6
4.3 Step 3 – The test statistic by hand . . . 6
Your turn . . . 7
4.4 Step 4 – The p-value . . . 7
Your turn . . . 8
4.5 Step 5 – Check in R and interpret . . . 8
Your turn . . . 9
4.6 Effect size: the raw difference . . . 9
4.7 A confidence interval for the raw difference . . . 9
4.8 Is the difference big? . . . 10
Your turn . . . 11
Your turn . . . 11

5 Two-sample t-test: comparing two groups **11**
5.1 First, describe the two groups . . . 12
Your turn . . . 13
5.2 Step 1 – Hypotheses . . . 13
Your turn . . . 13
5.3 Step 2 – Assumptions . . . 13
Your turn . . . 15
5.4 Step 3 – The test statistic by hand . . . 15
Your turn . . . 15
5.5 Step 4 & 5 – Check in R and interpret . . . 16
Your turn . . . 16
5.6 Effect size: Cohen's d (and checking it in R) . . . 16
Your turn . . . 18

6 Power, effect size, and sample size **18**
6.1 Type I and Type II errors . . . 18
6.2 A power simulation . . . 19
Your turn . . . 21
6.3 Statistical significance vs practical significance . . . 22
Your turn . . . 23

7 Recap: what you can now do **24**

How to use this tutorial

Welcome back! In Week 3 you learned to **estimate** a population mean: you computed a point estimate and a confidence interval, and you saw how the sampling distribution works.

This week we take one more step. Instead of asking "roughly what is the mean?", we ask a **yes/no question**: "is the population mean equal to a specific value, or not?" Answering that question is called **hypothesis testing**, and the tool we use is the **t-test**.

Work through this page carefully, from top to bottom. Every short explanation is followed by a **Your turn** task to do yourself.

General tips:

- Read each part slowly and in order. Each step builds on the one before it.
- Type the code yourself instead of copying and pasting it. You learn faster this way.
- Errors are normal. Read the message and try again.
- If you are stuck, ask a teacher or check the FAQ.
- Save your work often.

---

1 Start workflow

We begin, as always, with the **start workflow**. This week there is one extra step: we load the cleaned data you saved at the end of Week 2.

1. Open **RStudio**: click on the RStudio application.
2. Set the **working directory**: click *Session* > *Set Working Directory* > *Choose Directory*, and choose your `research_methods` folder.
3. Open a **new script**: click *File* > *New File* > *R Script*.
4. On the first line, write a comment with the title, for example: `# R tutorial - Week 4`.
5. Save the script as `week4.R` in your `research_methods` folder.
6. Load your cleaned data from Week 2 into an object called `survey`:

```
survey <- read.csv("class_survey_clean.csv")
```

> ⚠ Watch out
>
> If you see `Error: cannot open file ...`, R cannot find the file. Check that your working directory is `research_methods` (run `getwd()` to print it), and that `class_survey_clean.csv` really sits in that folder. If you do not have the file, go back to the end of the Week 2 tutorial and save it first.

Your turn

Complete the **start workflow** and load `survey`. Check that `survey` appears in the **Environment** pane (top-right).

---

2 From estimating to testing

In Week 3 you built a confidence interval: a **range** of plausible values for the population mean. That answered the question "what is the estimated mean?"

Sometimes, we have a different question. Suppose an article reports that, in some larger population, the average self-efficacy score is **30**. We can now ask: *does our class differ from that value?* This is a **yes/no** question about a specific number, and it is exactly what a **hypothesis test** answers.

The specific number we test against (here, 30) is called **mu** (written μ_0). It does not come from our own data; it comes from **theory or earlier research** (the article). Our job is to decide whether our observed data is consistent with that hypothesized value, or inconsistent.

i Example: turning a claim into a test

- **Claim from the literature:** the average self-efficacy is 30.
- **Our sample:** the students in our class.
- **Our question:** is our class's average self-efficacy *different* from 30?

We use a test that takes the wobble of the sample mean (the standard error from Week 3) into account.

---

3 The five steps of a hypothesis test

Every testing (e.g., t-tests) in this tutorial follows the same five steps. Learn them once, and apply them to any test. For the t-test we do the following:

i The five steps

1. **Hypotheses** – write down the null and the alternative.
2. **Assumptions** – check that a t-test is allowed for your data.
3. **Test statistic** – compute the t-value.
4. **p-value** – find how surprising that t-value is given your null hypothesis.
5. **Interpretation** – decide about the null hypothesis and say what it means.

We first walk these five steps for a **one-sample t-test** (comparing our mean to a known value). Then we repeat them for a **two-sample t-test** (comparing two group means). Finally, we explore **power** and **effect size**.

---

4 One-sample t-test: comparing our mean to a known value

4.1 Step 1 – Hypotheses

A hypothesis test always has two hypotheses.

- The **null hypothesis** (H_0) is the "no difference" statement: the population mean equals the value from the literature. We write $H_0 : \mu = 30$.
- The **alternative hypothesis** (H_1) is what we suspect instead: the population mean is **different** from 30. We write $H_1 : \mu \neq 30$.

Because H_1 uses "different" ($\neq$), and allows a difference in **either** direction (higher *or* lower), this is a **two-sided test**. Two-sided is our default, because usually we do not know in advance which way the difference will go.

> **i** Another form (optional): one-sided tests
>
> Sometimes you have a clear **direction** in mind *before* you see the data. For example, "our students score **higher** than 30." Then you can use a **one-sided test**, with $H_1 : \mu > 30$ (or $H_1 : \mu < 30$ for "lower").
> A one-sided test puts all of the 5% in one tail, which makes it a little easier to reach significance in that direction, but it means you **cannot** detect a difference in the other direction at all. In R you choose this with `alternative = "greater"` or `alternative = "less"` inside `t.test()`. In this course, use **two-sided** unless you have a strong, pre-registered reason to expect one direction.

Your turn

Pick a continuous variable and find a value for μ_0. Write down your H_0 and H_1 in the two-sided form ($=$ and $\neq$). In one sentence, explain why you chose that value for μ_0, and why you test two-sided.

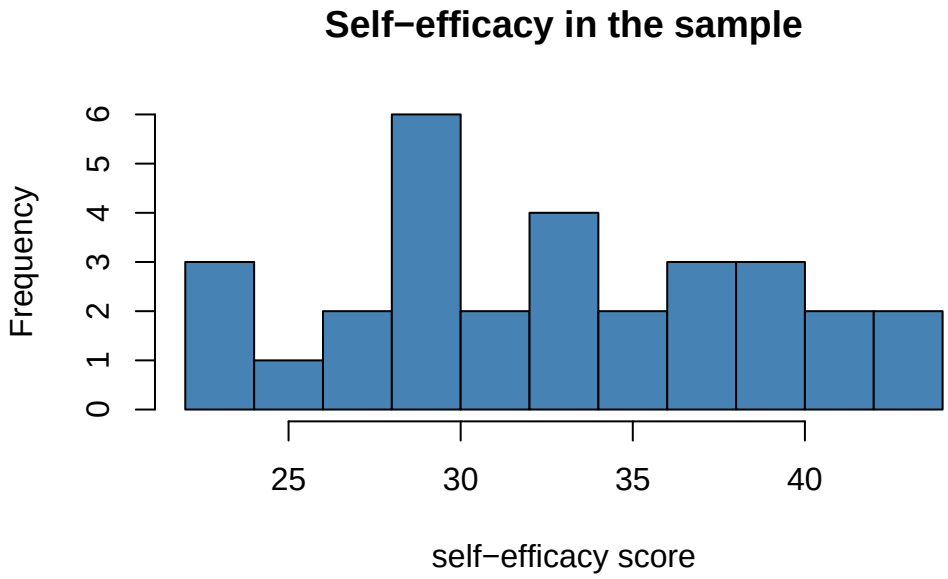
4.2 Step 2 – Assumptions

A one-sample t-test is only valid if a few **assumptions** hold:

- The variable is **continuous** (a real number, like a scale score).
- The observations are **independent**: one row per person, and people do not influence each other.
- The variable is roughly **normally distributed** in the population, *or* the sample is large enough that this matters little (thanks to the sampling distribution from Week 3, which becomes bell-shaped as the sample grows).

We check the shape with a histogram (from Week 2):

```r
hist(survey$self_efficacy,
     breaks = 10,
     col = "steelblue",
     main = "Self-efficacy in the sample",
     xlab = "self-efficacy score")
```



We look for a roughly symmetric, single-peaked shape. It does not have to be perfect, strong skew would be a warning sign.

Your turn

Make a histogram of your variable. Describe its shape in one or two sentences (symmetric or skewed? one peak or more? any outliers?). Argue whether the normality assumption is reasonable.

4.3 Step 3 – The test statistic by hand

The **test statistic** for a t-test is the **t-value**. It measures how far our sample mean sits from μ_0, expressed in **standard errors** (the same standard error you met in Week 3):

$$t = \frac{\text{mean} - \mu_0}{\text{standard error}}, \qquad \text{standard error} = \frac{sd}{\sqrt{n}}$$

A t-value near 0 means the sample mean is close to μ_0. A large t-value (far from 0, positive or negative) means the sample mean is far from μ_0.

We compute it step by step in R, as a calculator:

```
mu0 <- 30                              # the value from the literature

m  <- mean(survey$self_efficacy)       # sample mean
s  <- sd(survey$self_efficacy)         # standard deviation
n  <- nrow(survey)                     # number of people
se <- s / sqrt(n)                      # standard error

t_value <- (m - mu0) / se
t_value
#> [1] 2.93013
```

Your turn

Compute the t-value **by hand** (in R, as a calculator) for your own variable and your own μ_0, following the four lines above: the mean, the standard deviation, the standard error, and finally the t-value.

4.4 Step 4 – The p-value

The **p-value** answers: *if the null hypothesis were true, how surprising would our t-value be?* A **small** p-value (close to 0) means "this observed result would be very surprising if H_0 were true", which is treated as evidence **against** H_0. The usual yet arbitrary cut-off is **0.05**.

The t-value follows a **t-distribution** with `n - 1` **degrees of freedom** (`df`). For a **two-sided** test, we look at both tails, so we take the area beyond $+|t|$ and beyond $-|t|$:

```
df <- n - 1
p_value <- 2 * pt(-abs(t_value), df = df)   # two-sided p-value
p_value
#> [1] 0.006541451
```

The function `pt()` gives the area under the t-distribution up to a value. Using `-abs(t_value)` takes the lower tail, and multiplying by 2 covers both sides.

> **By hand with a distribution table**
>
> On paper, you do not have `pt()`. Instead you look up your `df` and your $|t|$ in a **distribution table** (in the book). The table gives a one-sided area; for a **two-sided** test you **double** it. Always check whether your table is one-sided or two-sided before you read off the p-value.

Your turn

Compute the two-sided p-value for your variable with `pt()`, using your own `t_value` and `df`. Is it smaller or larger than 0.05?

4.5 Step 5 – Check in R and interpret

You did the work by hand. Now let R do the whole test in one line and check your answer. The command is `t.test()`. We give it the variable and the value μ_0 under `mu`:

```
t.test(survey$self_efficacy, mu = 30)   # default is alternative = "two.sided"
#>
#>  One Sample t-test
#>
#> data:  survey$self_efficacy
#> t = 2.9301, df = 29, p-value = 0.006541
#> alternative hypothesis: true mean is not equal to 30
#> 95 percent confidence interval:
#>  30.97647 35.49020
#> sample estimates:
#> mean of x
#>  33.23333
```

Read the output: it prints the **t** value, the **df**, and the **p-value**. These should match your hand calculation from Steps 3 and 4. It also prints a **95% confidence interval** for the mean, exactly like Week 3.

> **Testing and confidence intervals are two sides of one coin**
>
> Look at the 95% confidence interval in the output. A two-sided test at the 0.05 level **rejects** H_0 when μ_0 (here 30) falls **outside** that interval, and **fails to reject** when μ_0 falls **inside** it. So the Week 3 confidence interval already contained the answer to this test.

Interpretation. We compare the p-value to 0.05:

- If $\mathbf{p < 0.05}$, we **reject** H_0: there is evidence that the population mean differs from μ_0.
- If $\mathbf{p} \geq 0.05$, we **do not reject** H_0: we do not have enough evidence to say it differs from μ_0. (This is *not* the same as proving H_0 true.)

Your turn

Run `t.test()` for your variable with your μ_0. Confirm that the `t` and `p` match your hand values. Then write one sentence stating your conclusion about H_0 in plain language.

4.6 Effect size: the raw difference

A p-value tells you *whether* there is a difference, but not *how big* it is. With a large enough sample, even a tiny, unimportant difference can become "significant". So we also report an **effect size**, which measures the size of the difference on its own.

The simplest effect size is the **raw difference** (also called the **unstandardized mean difference**). It is the distance between the sample mean and μ_0, in the **original units** of your variable, for example points on a questionnaire or hours per week:

$$\text{raw difference} = \text{mean} - \mu_0$$

```
raw_diff <- m - mu0
raw_diff
#> [1] 3.233333
```

The sign tells you the **direction**. A positive raw difference means the sample mean lies **above** μ_0. A negative raw difference means it lies **below** μ_0.

4.7 A confidence interval for the raw difference

The raw difference is an estimate, so it shows sampling fluctuation, just like the sample mean in Week 3. We therefore also give a 95% **confidence interval** for it. We build it in the same way as in Week 3, with one change: instead of 1.96 we use a multiplier from the **t-distribution**. The function `qt()` finds it for you. It is the t-distribution version of `qnorm()` from Week 3.

```r
se_diff <- s / sqrt(n)                # standard error (Week 3)
t_crit  <- qt(0.975, df = n - 1)      # t-multiplier for a 95% interval
c(raw_diff - t_crit * se_diff,
  raw_diff + t_crit * se_diff)        # 95% CI for the raw difference
#> [1] 0.9764682 5.4901985
```

If the interval contains **0**, then "no difference" is still a plausible value, and the two-sided test is not significant at 0.05. If 0 lies outside the interval, the test is significant.

4.8 Is the difference big?

There are **no fixed labels** such as "small" or "large" for a raw difference. A difference of 2 points can be large on one scale and tiny on another. You have to judge the size yourself, **in context**. These questions help:

1. **How long is the scale?** A difference of 2 points is large on a scale from 0 to 10, but small on a scale from 0 to 100.
2. **What does it mean in practice?** For example: is one extra hour of study per week something a teacher would notice?
3. **Is there a known important difference?** Some test manuals say how many points a change must be before it matters for a child or client.
4. **Look at both ends of the confidence interval.** Is even the **smallest** plausible difference important? Or is even the **largest** plausible difference still unimportant?

i Example: judging a raw difference

Imagine a self-efficacy scale that runs from 10 to 50, with $\mu_0 = 30$. The sample of 30 students has a mean of 32.4. The raw difference is $32.4 - 30 = 2.4$ points, with a 95% confidence interval from 0.2 to 4.6 points.
A conclusion could be: "On average, the students scored 2.4 points higher than 30 (95% CI from 0.2 to 4.6 points). The difference is statistically significant, but it is small compared to the 40-point length of the scale. The smallest plausible difference (0.2 points) is negligible; the largest (4.6 points) might be noticeable in practice."

i Why the raw difference and t are not the same

The t-value is the raw difference divided by the standard error:

$$t = \frac{\text{mean} - \mu_0}{sd/\sqrt{n}}$$

A bigger sample makes the standard error smaller (Week 3), so the t-value **grows** with

the sample size. The raw difference does **not** grow with the sample size. That is the whole point: the raw difference describes the size of the effect itself, separately from how many people you measured. We return to this idea in the power section.

Your turn

For your own variable and μ_0:

1. Compute the raw difference. Is the sample mean above or below μ_0?
2. Compute the 95% confidence interval for the raw difference. Does it contain 0? Does this agree with your p-value?
3. Answer the four questions in *Is the difference big?* in one sentence each.
4. Write one or two sentences that combine your Step 5 conclusion with the raw difference and its confidence interval, in the units of your variable.

Your turn

Think it through. Two researchers measure stress in parents, on a scale from 0 to 40. Both compare their sample mean with the same μ_0.

- Researcher A asked **20** parents. The raw difference is **4 points**, with $p = 0.08$.
- Researcher B asked **2000** parents. The raw difference is **0.8 points**, with $p < 0.001$.

1. Which study found the **larger** effect?
2. Explain why the smaller effect is significant and the larger one is not. Use the words **sample size** and **standard error**.
3. Which result do you think is more important for practice? Explain your answer.
4. What would you advise researcher A to do next?

---

5 Two-sample t-test: comparing two groups

Often we do not compare our mean to a fixed number, but compare **two groups** to each other. For example: *do students in different study programmes report different self-efficacy?* This is a **two-sample t-test**. It follows the same five steps.

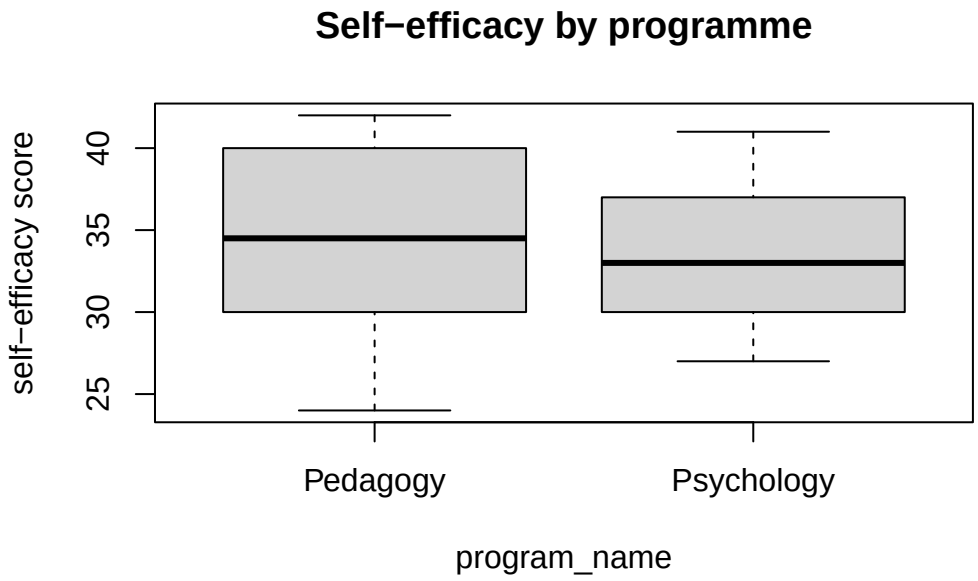
For this example we compare two of the programmes in our data, **Pedagogy** and **Psychology**. We first keep only those two groups:

```r
two_programs <- survey[survey$program_name %in% c("Pedagogy", "Psychology"), ]
```

5.1 First, describe the two groups

Before testing, always look at the groups. We split self-efficacy into the two programmes and compare them with a boxplot (from Week 2) and with numbers:

```r
boxplot(self_efficacy ~ program_name,
        data = two_programs,
        main = "Self-efficacy by programme",
        ylab = "self-efficacy score")
```




```r
# the two groups as separate vectors
g1 <- two_programs$self_efficacy[two_programs$program_name == "Pedagogy"]
g2 <- two_programs$self_efficacy[two_programs$program_name == "Psychology"]

c(mean_pedagogy = mean(g1), sd_pedagogy = sd(g1), n_pedagogy = length(g1))
#> mean_pedagogy   sd_pedagogy    n_pedagogy
#>     33.900000      6.226288     10.000000
c(mean_psychology = mean(g2), sd_psychology = sd(g2), n_psychology = length(g2))
```

```
#> mean_psychology   sd_psychology    n_psychology
#>       33.555556        4.693376        9.000000
```

Your turn

Make the boxplot and the group summaries for your own two groups. Which group has the higher mean in the sample? Is the difference big or small compared to the spread within each group?

5.2 Step 1 – Hypotheses

Now the hypotheses are about the **difference** between the two group means.

- H_0: the two population means are equal (their difference is 0). Written $H_0 : \mu_1 = \mu_2$.
- H_1: the two population means are different. Written $H_1 : \mu_1 \neq \mu_2$.

Again this is **two-sided**: we allow either group to be higher.

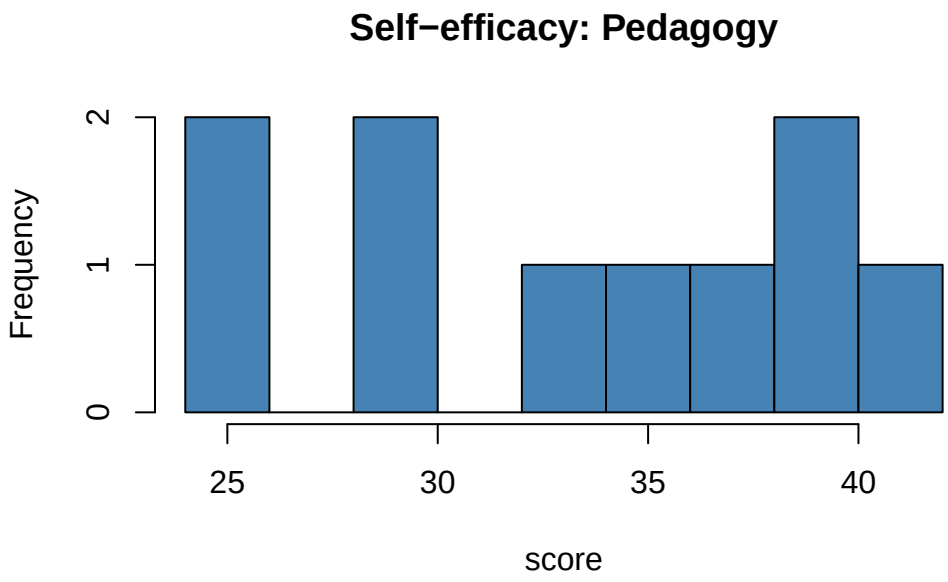
Your turn

Write down H_0 and H_1 for your two groups, in the two-sided form. Explain in one sentence why you test two-sided.

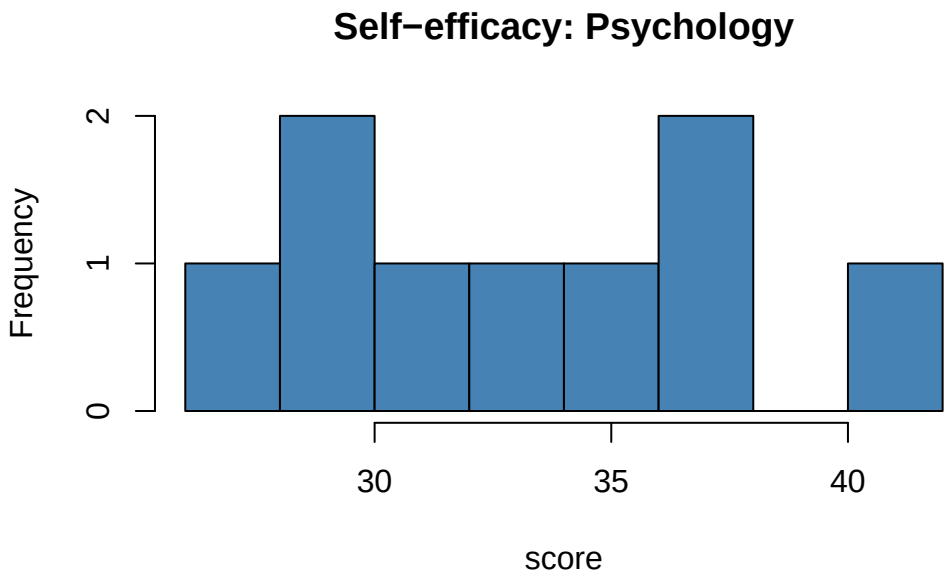
5.3 Step 2 – Assumptions

The two-sample t-test needs the same assumptions as before, applied to **both** groups: a continuous variable, independent observations, and roughly normal distributions in each group (or large enough samples). Check each group's shape with a histogram:

```
hist(g1, breaks = 8, col = "steelblue",
     main = "Self-efficacy: Pedagogy", xlab = "score")
```




```
hist(g2, breaks = 8, col = "steelblue",
     main = "Self-efficacy: Psychology", xlab = "score")
```



Your turn

Make a histogram for each of your two groups. Describe both shapes and argue whether the assumption is reasonable.

5.4 Step 3 – The test statistic by hand

For two groups, the t-value is the **difference between the two means**, divided by the **standard error of that difference**. Each group keeps its **own** spread: we do **not** assume the two groups are equally spread out. This version of the test is called **Welch's t-test**, and it is the safer, standard choice.

$$t = \frac{\text{mean}_1 - \text{mean}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

In R, as a calculator:

```
m1 <- mean(g1); s1 <- sd(g1); n1 <- length(g1)
m2 <- mean(g2); s2 <- sd(g2); n2 <- length(g2)

# standard error of the difference (each group keeps its own spread)
se_diff  <- sqrt(s1^2/n1 + s2^2/n2)
t_value2 <- (m1 - m2) / se_diff
t_value2
#> [1] 0.1369672
```

The **degrees of freedom** for Welch's test use a longer formula. You do not need to memorise it; R computes it for you, and we reproduce it here only so our hand p-value matches R exactly:

```
df2 <- (s1^2/n1 + s2^2/n2)^2 /
       ((s1^2/n1)^2 / (n1 - 1) + (s2^2/n2)^2 / (n2 - 1))
df2
#> [1] 16.53635
```

Your turn

Compute the two-sample t-value **by hand** for your two groups, following the lines above: the two means, the standard error of the difference, and the t-value.

5.5 Step 4 & 5 – Check in R and interpret

First, the two-sided **p-value** by hand, using `pt()` with the Welch degrees of freedom:

```
p_value2 <- 2 * pt(-abs(t_value2), df = df2)
p_value2
#> [1] 0.8927105
```

Now let R run the whole test and check your hand calculation. For two groups we use the **formula form** `variable ~ group` (as in the Week 2 boxplot). We do **not** set `var.equal`, so `t.test()` runs **Welch's test** by default, matching the by-hand calculation above:

```
t.test(self_efficacy ~ program_name, data = two_programs)
#>
#>  Welch Two Sample t-test
#>
#> data:  self_efficacy by program_name
#> t = 0.13697, df = 16.536, p-value = 0.8927
#> alternative hypothesis: true difference in means between group Pedagogy and group Psychol
#> 95 percent confidence interval:
#>  -4.972666  5.661554
#> sample estimates:
#>   mean in group Pedagogy mean in group Psychology
#>                 33.90000                 33.55556
```

The `t`, `df`, and `p-value` should match your hand values. Read the p-value and decide:

- $\mathbf{p < 0.05}$ → reject H_0: the two programmes differ in self-efficacy.
- $\mathbf{p} \geq 0.05$ → do not reject H_0: not enough evidence of a difference.

Your turn

Compute the two-sided p-value by hand, then run the two-sample `t.test()` for your groups. Confirm the `t` and `p` match, and write one sentence stating your conclusion about H_0.

5.6 Effect size: Cohen's d (and checking it in R)

For two groups, **Cohen's d** is the difference between the group means divided by a single standard deviation that combines both groups (the **pooled** standard deviation, `sp`). This pooling is only a way to put the difference on one common scale for the effect size; it is **not** an assumption of the Welch test above.

We subtract in the order (Psychology − Pedagogy), that is $\text{mean}_2 - \text{mean}_1$, so that the sign matches what `cohen.d()` reports below:

$$d = \frac{\text{mean}_2 - \text{mean}_1}{s_p}, \qquad s_p = \sqrt{\frac{(n_1 - 1)\, s_1^2 + (n_2 - 1)\, s_2^2}{n_1 + n_2 - 2}}$$

```
sp <- sqrt(((n1 - 1) * s1^2 + (n2 - 1) * s2^2) / (n1 + n2 - 2))

cohens_d2 <- (m2 - m1) / sp
cohens_d2
#> [1] -0.06197456
```

Use the same rough labels as before (0.2 small, 0.5 medium, 0.8 large). The **sign** of d only shows the direction (which group is higher); its **size** is what matters.

You can also let a package compute it. The `psych` package has a function `cohen.d()`. The block below installs `psych` the first time (only if it is missing) and then loads it:

```
if (!requireNamespace("psych", quietly = TRUE)) {
  install.packages("psych", repos = "https://cloud.r-project.org")
}
library(psych)

d_data <- data.frame(
  self_efficacy = two_programs$self_efficacy,
  program_name  = factor(two_programs$program_name)
)
cohen.d(d_data, group = "program_name")
#> Call: cohen.d(x = d_data, group = "program_name")
#> Cohen d statistic of difference between two means
#>               lower effect upper
#> self_efficacy -0.97  -0.07  0.84
#>
#> Multivariate (Mahalanobis) distance between groups
#> [1] 0.066
#> r equivalent of difference between two means
#> self_efficacy
#>         -0.03
```

The value from `cohen.d()` should be **very close** to your hand value (small differences can appear because the package estimates the combined spread in a slightly different way).

i A note on signs

`t.test()` and `cohen.d()` order the two groups in opposite ways, so the **t-value** and **Cohen's d** may carry opposite signs. That is harmless: only the *sizes* matter for your conclusion. The sign simply tells you which group scored higher.

Your turn

Compute Cohen's d by hand for your two groups and label it. Then check it with `cohen.d()` from the `psych` package and confirm the two are close. Finally, write a short conclusion that combines the p-value **and** the effect size.

---

6 Power, effect size, and sample size

A test can give the wrong answer in two ways. Understanding these leads to the idea of **power**.

6.1 Type I and Type II errors

There are two kinds of mistake a test can make:

	H_0 is really true	H_0 is really false
Test rejects H_0	Type I error (false alarm)	Correct
Test keeps H_0	Correct	Type II error (missed effect)

- A **Type I error** is a **false alarm**: we reject H_0 when it is actually true. Its rate is α (alpha), the 0.05 cut-off we choose.
- A **Type II error** is a **missed effect**: we keep H_0 when there really is a difference.
- **Power** is the chance of **correctly** detecting a real effect. It equals $1-$(Type II error rate). High power is good.

Power goes **up** when the effect is bigger, when the sample is larger, or when the spread is smaller.

6.2 A power simulation

We can *see* power by simulation, using a prescribed function. It draws many samples from a population whose true mean we choose, runs a one-sample t-test on each, and reports the **share of tests that are significant**. You can run and read this; you do **not** need to write it yourself.

```
simulate_power <- function(mu0, true_mean, true_sd, n, n_sim = 1000, alpha = 0.05) {
  set.seed(2026)
  p_values <- replicate(n_sim, {
    sample_data <- rnorm(n, mean = true_mean, sd = true_sd)
    t.test(sample_data, mu = mu0)$p.value
  })
  mean(p_values < alpha)   # share of tests that reject H0
}
```

First, a check. If the true mean **equals** μ_0, then H_0 is true, so every rejection is a false alarm. The share should land near $\alpha = 0.05$ (the Type I error rate):

```
simulate_power(mu0 = 30, true_mean = 30, true_sd = 6, n = 30)   # H0 true -> about 0.05
#> [1] 0.044
```

Now make the true mean **differ** from μ_0. The share of significant tests is now the **power**:

```
simulate_power(mu0 = 30, true_mean = 33, true_sd = 6, n = 30)   # a real 3-point effect
#> [1] 0.762
```

Whatever is left over (1 minus this number) is the **Type II error rate**: the share of times we missed the real effect.

What power looks like. The picture below shows *why* this works. The grey curve is the sampling distribution of the mean if H_0 were true (centred on 30). The blue curve is the sampling distribution if the true mean is really 33. The red dashed lines are the cut-offs: sample means outside them lead us to reject H_0. **Power** is the share of the blue curve that falls beyond the cut-off (shaded):

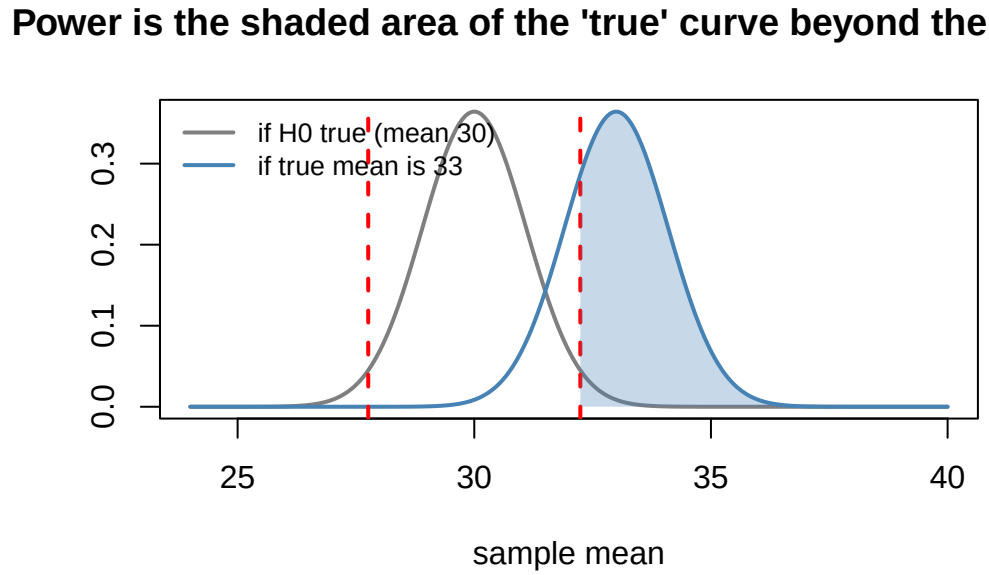
```
mu0       <- 30
true_mean <- 33
true_sd   <- 6
n         <- 30
se        <- true_sd / sqrt(n)
```

```r
crit      <- qt(0.975, df = n - 1) * se   # half-width of the "keep H0" zone
upper_cut <- mu0 + crit
lower_cut <- mu0 - crit

xs <- seq(24, 40, length.out = 400)
plot(xs, dnorm(xs, mu0, se), type = "l", lwd = 2, col = "grey50",
     xlab = "sample mean", ylab = "",
     main = "Power is the shaded area of the 'true' curve beyond the cut-off")
lines(xs, dnorm(xs, true_mean, se), lwd = 2, col = "steelblue")
abline(v = c(lower_cut, upper_cut), col = "red", lwd = 2, lty = 2)

# shade the power region (blue curve beyond the upper cut-off)
xr <- seq(upper_cut, 40, length.out = 200)
polygon(c(xr, rev(xr)),
        c(dnorm(xr, true_mean, se), rep(0, length(xr))),
        col = adjustcolor("steelblue", alpha.f = 0.3), border = NA)

legend("topleft", bty = "n", cex = 0.85, lwd = 2,
       col = c("grey50", "steelblue"),
       legend = c("if H0 true (mean 30)", "if true mean is 33"))
```



If the two curves move further apart (a bigger effect) or get narrower (a bigger sample), more of the blue curve spills past the cut-off, so power goes up.

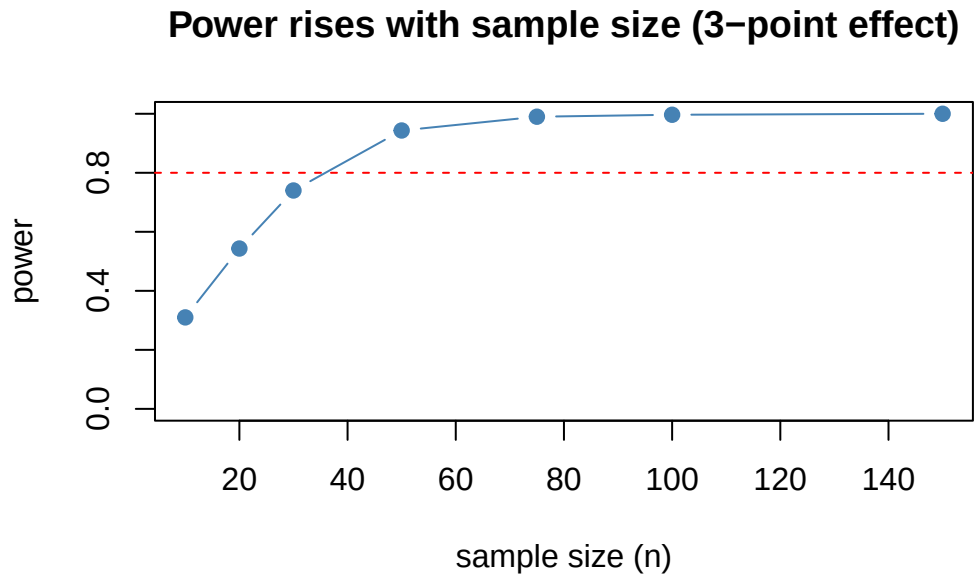
Your turn

Keep `mu0 = 30`, `true_mean = 33`, `true_sd = 6`, and raise the sample size `n` from 30 to 100 in `simulate_power()`. Does the power go up or down? Write one sentence linking sample size to power.

We can watch that happen across many sample sizes at once. For each `n` we run the simulation and plot the resulting power:

```
sample_sizes <- c(10, 20, 30, 50, 75, 100, 150)
power_by_n <- sapply(sample_sizes, function(nn) {
  simulate_power(mu0 = 30, true_mean = 33, true_sd = 6, n = nn, n_sim = 300)
})

plot(sample_sizes, power_by_n, type = "b", pch = 19, col = "steelblue",
     ylim = c(0, 1),
     xlab = "sample size (n)", ylab = "power",
     main = "Power rises with sample size (3-point effect)")
abline(h = 0.80, col = "red", lty = 2)   # a common target for power
```



The curve climbs as `n` grows, and the red line marks 0.80, a common target: many researchers aim for at least 80% power.

> **Going further (optional): power.t.test()**
>
> R also has a built-in function, `power.t.test()`, that computes power from the effect size, sample size, and spread directly, without simulation. Our simulation is slower but shows *where* power comes from, which is easier to understand.

6.3 Statistical significance vs practical significance

Here is the most important lesson of the week. Because power rises with sample size, a **very large sample** can make **any** difference significant, even one far too small to matter in practice.

Take a tiny true difference: $\mu_0 = 30$ versus a true mean of 30.3. In standard deviations that is Cohen's d = 0.3 / 6 = **0.05**, a trivially small effect. With a normal sample it is almost never detected:

```
simulate_power(mu0 = 30, true_mean = 30.3, true_sd = 6, n = 30)     # tiny effect, small n
#> [1] 0.054
```

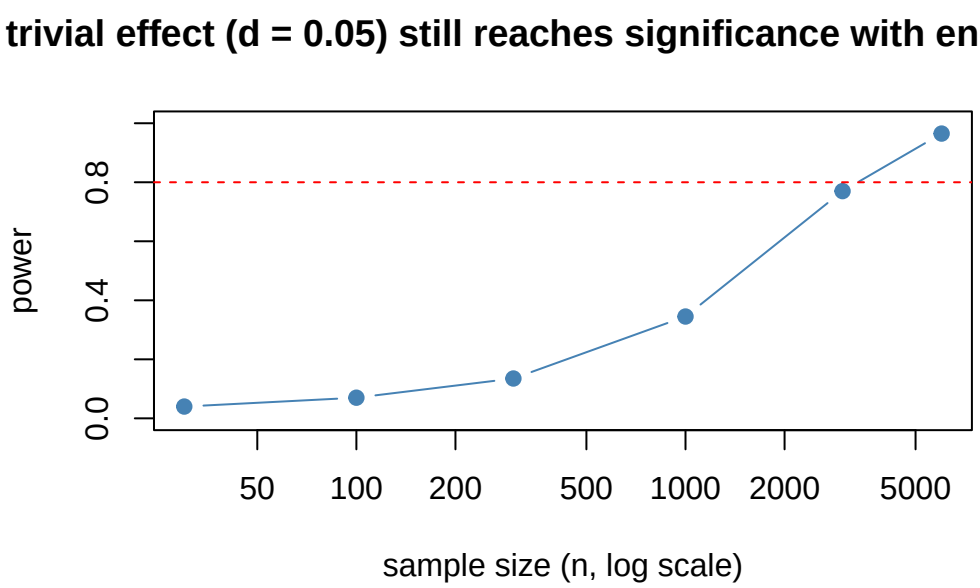
But with a huge sample, the same trivial difference becomes significant almost every time:

```
simulate_power(mu0 = 30, true_mean = 30.3, true_sd = 6, n = 5000)   # tiny effect, huge n
#> [1] 0.942
```

We can plot this trivial effect across a wide range of sample sizes. Watch the power climb toward 1 even though the effect never changes:

```
big_sizes <- c(30, 100, 300, 1000, 3000, 6000)
power_tiny <- sapply(big_sizes, function(nn) {
  simulate_power(mu0 = 30, true_mean = 30.3, true_sd = 6, n = nn, n_sim = 200)
})

plot(big_sizes, power_tiny, type = "b", pch = 19, col = "steelblue",
     log = "x", ylim = c(0, 1),
     xlab = "sample size (n, log scale)", ylab = "power",
     main = "A trivial effect (d = 0.05) still reaches significance with enough data")
abline(h = 0.80, col = "red", lty = 2)
```



So a small p-value does **not** mean the effect is important. It can simply mean the sample was very large. This is the difference between:

- **Statistical significance:** the p-value is below 0.05 (the effect is unlikely to be pure chance).
- **Practical significance:** the effect is **large enough to matter** in the real world.

This is exactly why we always report **Cohen's d next to the p-value**. The p-value tells you *whether* there is an effect; the effect size tells you *whether it matters*.

Your turn

Use `simulate_power()` twice: once with a **small effect and a large n**, and once with a **large effect and a small n**. Compare the two power values. Then write two sentences: one on why a significant result is not always an important result, and one on why reporting Cohen's d helps.

---

7 Recap: what you can now do

Last week you estimated a population mean. This week you learned to **test** claims about it. You can now:

- write a **null** and **alternative** hypothesis, and choose two-sided (or, rarely, one-sided);
- check the **assumptions** of a t-test with `hist()`;
- compute a **one-sample t-value** by hand as `(mean - mu0) / se`;
- find a two-sided **p-value** with `pt()` (and look it up in a distribution table);
- run and read `t.test(x, mu = ...)` and interpret the result;
- compare **two groups** with a two-sample t-test (Welch), by hand and with `t.test(y ~ group)`;
- compute **Cohen's d** for one and two samples, and check it with `psych::cohen.d()`;
- explain **power**, **Type I** and **Type II** errors, and simulate and plot them;
- explain why **statistical significance is not the same as practical significance**.

A small cheat sheet to keep:

You want to…	You write…
Sample mean / sd / n	`mean(x)`, `sd(x)`, `length(x)`
Standard error	`sd(x) / sqrt(length(x))`
One-sample t by hand	`(mean(x) - mu0) / se`
Two-sided p-value	`2 * pt(-abs(t), df = n - 1)`
One-sample t-test	`t.test(x, mu = 30)`
One-sided t-test	`t.test(x, mu = 30, alternative = "greater")`
Two-sample t-test (Welch)	`t.test(y ~ group, data = d)`
Cohen's d (one sample)	`(mean(x) - mu0) / sd(x)`
Cohen's d (two groups)	`(m2 - m1) / sp`
Cohen's d in R	`psych::cohen.d(d, group = "g")`
Simulate many p-values	`replicate(1000, t.test(rnorm(n, ...), mu = )$p.value)`

Well done. Keep your materials in the `research_methods` folder, we build directly on them next week.
